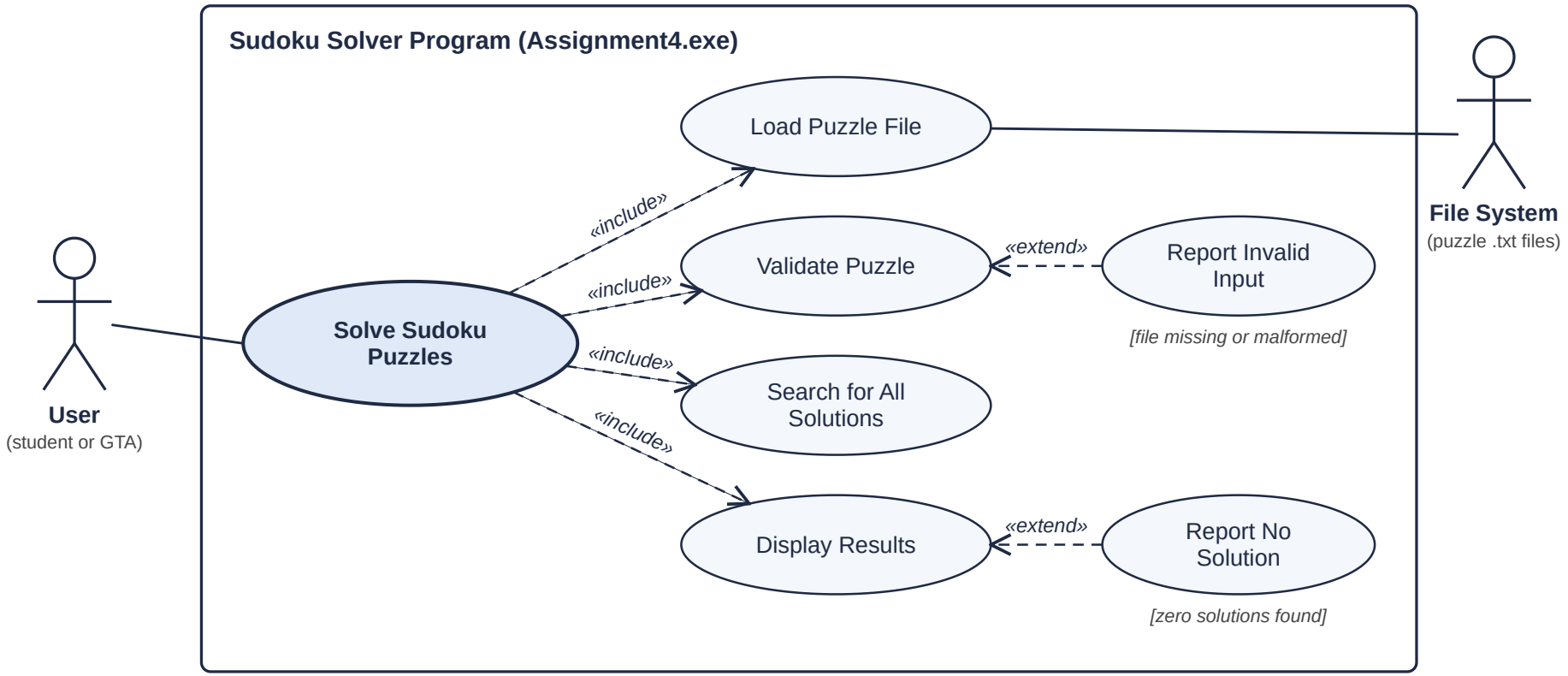
EECS 348 Assignment 4 — UML Use Case Model: Sudoku Solver

Erick Marte · October 15, 2026 · Prepared with assistance from Claude (Anthropic, claude.ai)

1. Identified Use Case — Use Case Diagram



Primary use case: Solve Sudoku Puzzles. **Primary actor:** User. **Secondary actor:** File System. **«include»**: behavior that is always part of the base use case. **«extend»**: optional behavior that runs only under the condition shown in brackets.

2. Brief Description

Solve Sudoku Puzzles. The User runs the Sudoku solver on one or more puzzle files. For each file, the system reads and validates the puzzle, prints the file name and the puzzle, and then uses a recursive depth-first search with backtracking to find *every* solution. It prints each solution as it is found, or "No solution found" when none exists, followed by the total number of solutions. Malformed files are reported with an error message and skipped.

3. Step-by-Step Flow of Events (Outline)

1. The User starts the program (`./Assignment4.exe`).
2. The system selects the files to process: `puzzle1.txt` through `puzzle5.txt` , or the file names the User typed after the command.
3. For each puzzle file, the system:
    a. prints the puzzle file name;
    b. reads the puzzle from the file (*Load Puzzle File*);
    c. checks the file format and the given digits (*Validate Puzzle*);
    d. prints the initial puzzle;
    e. searches for all solutions with recursive depth-first search and backtracking (*Search for All Solutions*);
    f. prints each solution as it is found, then the total number of solutions (*Display Results*).
4. After the last file, the program ends.

Exceptions: if a file is missing or malformed at step 3b–3c, the system prints an “Invalid input” message and moves to the next file (*Report Invalid Input*). If no solution exists at step 3e, the system prints “No solution found” (*Report No Solution*).

4. Detailed Description

Use case name	Solve Sudoku Puzzles
Use case ID	UC-1
Primary actor	User — the student or GTA who runs `Assignment4.exe` on an EECS cycle server.
Secondary actor	File System — supplies the puzzle text files from the current directory.
Stakeholders & interests	• **User:** wants every correct solution printed clearly, and an understandable message when a puzzle cannot be solved or a file is bad. • **Grader (GTA):** wants output that shows the file name, the puzzle, and all solutions or “No solution found”, as the assignment specifies.
Trigger	The User types `./Assignment4.exe` (optionally followed by file names) at the command prompt.
Preconditions	• `Assignment4.exe` has been compiled on a cycle server. • The puzzle files are in the current working directory, or their paths are given on the command line.

Main success scenario (basic flow)	1. The User runs `./Assignment4.exe` with no arguments. 2. The system chooses `puzzle1.txt` … `puzzle5.txt` as the files to process. 3. The system prints a header containing `Puzzle File: <name>`. 4. The system opens the file and reads it one line at a time, ignoring blank lines. 5. The system checks that there are exactly 9 non-blank lines, that each has exactly 9 entries, and that each entry is a digit 1–9 or `_` (a blank square). 6. As it stores each given digit, the system records it in that row's, column's, and 3×3 box's set of used digits, and confirms that no given digit repeats. 7. The system prints `Initial Puzzle:` followed by the 9×9 grid. 8. The system searches recursively: it picks the empty cell with the fewest legal digits, places each legal digit in turn, and calls itself on the rest of the board. When a cell has no legal digit (a dead end), it backtracks by removing the last digit and trying the next one. 9. Each time no empty cell remains, the system prints `Solution n:` and the completed grid, then keeps backtracking to look for more solutions. 10. When every possibility has been tried, the system prints `Total solutions found: n`. 11. Steps 3–10 repeat for each remaining puzzle file. 12. The program ends with exit status 0.
Alternative flows	**A1 – File names supplied (step 1).** The User types file names after the command. The system processes those files instead of the five defaults, then continues at step 3. **A2 – File cannot be opened (step 4).** The system prints `Invalid input: could not open <name>`, remembers to exit with status 1, and continues at step 11. **A3 – File is malformed (step 5).** A line has more or fewer than 9 entries, there are not exactly 9 rows, or an entry is not 1–9 or `_`. The system prints `Invalid input:` with the reason and line number, remembers to exit with status 1, and continues at step 11. **A4 – Given digits break a rule (step 6).** The system still prints the puzzle (step 7) but skips the search. It prints `No solution found` and the reason (which digit repeats, with its row and column), then `Total solutions found: 0`, and continues at step 11. **A5 – No solution exists (step 9).** The search ends without ever completing the board. The system prints `No solution found` and `Total solutions found: 0` (for example, `puzzle4.txt`). **A6 – More than one solution (step 9).** The system prints every solution, numbered in the order found (for example, `puzzle5.txt` has two).

Postconditions	• **Success:** for every file, the file name, the initial puzzle, and every solution (or "No solution found") have been printed, followed by the solution count. The puzzle files are not changed. • **Failure:** each missing or malformed file has been reported with an error message, the remaining files have still been processed, and the program exits with status 1.
Special requirements	• Written in C++ as an object-oriented program (a `SudokuSolver` class). • Solutions are found with a recursive depth-first search with backtracking. • Compiles and runs on the EECS cycle servers (`g++ -std=c++11`); all output goes to standard output.
Assumptions	Puzzles are 9×9 and use the same text format as the provided puzzle files: digits and underscores separated by spaces, one row per line.